

Nama: Izzatus Shofwa Ar Rafei

NIM: 25091397038

Kelas: 2025B

Penjelasan Praktikum

1. Modified Binary Search

```
1 def countOccurrences(sortedList, target):
2
3     # mencari kemunculan pertama
4     def findFirst():
5         left, right = 0, len(sortedList) - 1
6         first = -1
7
8         while left <= right:
9             mid = (left + right) // 2
10
11             if sortedList[mid] == target:
12                 first = mid
13                 right = mid - 1
14             elif sortedList[mid] < target:
15                 left = mid + 1
16             else:
17                 right = mid - 1
18
19         return first
20
```

```
21     # mencari kemunculan terakhir
22     def findLast():
23         left, right = 0, len(sortedList) - 1
24         last = -1
25
26         while left <= right:
27             mid = (left + right) // 2
28
29             if sortedList[mid] == target:
30                 last = mid
31                 left = mid + 1
32             elif sortedList[mid] < target:
33                 left = mid + 1
34             else:
35                 right = mid - 1
36
37         return last
38
39     first = findFirst()
40     last = findLast()
41
42     if first == -1:
43         return 0
44
45     return last - first + 1
46
47
48 # contoh
49 print(countOccurrences([1,2,4,4,4,4,7,9,12],4))
50 print(countOccurrences([1,2,4,4,4,4,7,9,12],5))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\semester 2 shofwa\struktur data> & C:/Users/DELL/AppDa
2 shofwa/struktur data/modified_binary_search.py"
4
0
PS D:\semester 2 shofwa\struktur data>
```

Analisis Hasil: Kode tersebut berfungsi untuk menghitung berapa kali sebuah nilai target muncul dalam list yang sudah terurut dengan menggunakan metode binary search agar proses pencarian lebih cepat. Di dalam fungsi terdapat dua proses, yaitu mencari posisi kemunculan pertama dan posisi kemunculan terakhir dari target dengan membagi area pencarian menggunakan variabel left, right, dan mid. Setelah kedua posisi ditemukan, jumlah kemunculan dihitung dengan rumus last - first + 1, sedangkan jika target tidak ditemukan maka fungsi akan mengembalikan nilai 0. Ketika program dijalankan, pemanggilan `countOccurrences([1,2,4,4,4,4,7,9,12],4)` menghasilkan 4 karena angka 4 muncul sebanyak empat kali, sedangkan `countOccurrences([1,2,4,4,4,4,7,9,12],5)` menghasilkan 0 karena angka 5 tidak terdapat di dalam list tersebut.

2. Bubble Sort dengan Analisis Langkah

```
bubble_sort_dengan_analisis_langkah.py > bubbleSort
1 def bubbleSort(arr):
2     arr = arr.copy()
3     n = len(arr)
4
5     total_comparisons = 0
6     total_swaps = 0
7     passes_used = 0
8
9     for i in range(n-1):
10         swapped = False
11         passes_used += 1
12
13         for j in range(n-i-1):
14             total_comparisons += 1
15
16             if arr[j] > arr[j+1]:
17                 arr[j], arr[j+1] = arr[j+1], arr[j]
18                 total_swaps += 1
19                 swapped = True
20
21         print(f"Pass {passes_used}: {arr}")
22
23         if not swapped:
24             break
25
26     return arr, total_comparisons, total_swaps, passes_used
27
28
29 # uji input
30 print("\nInput 1")
31 result1 = bubbleSort([5,1,4,2,8])
32 print("Hasil:", result1)
33
34 print("\nInput 2")
35 result2 = bubbleSort([1,2,3,4,5])
36 print("Hasil:", result2)
```

```
Input 1
Pass 1: [1, 4, 2, 5, 8]
Pass 2: [1, 2, 4, 5, 8]
Pass 3: [1, 2, 4, 5, 8]
Hasil: ([1, 2, 4, 5, 8], 9, 4, 3)

Input 2
Pass 1: [1, 2, 3, 4, 5]
Hasil: ([1, 2, 3, 4, 5], 4, 0, 1)
```

Analisis Hasil: Program membandingkan dua elemen yang bersebelahan lalu menukarnya jika urutannya salah, serta mencatat jumlah perbandingan, jumlah pertukaran, dan jumlah pass yang dilakukan. Setiap pass juga menampilkan kondisi list saat itu. Jika dalam satu pass tidak terjadi pertukaran, maka proses berhenti karena list sudah terurut. Saat dijalankan dengan input [5,1,4,2,8], program melakukan beberapa pass hingga menghasilkan list terurut [1,2,4,5,8]. Sedangkan pada input [1,2,3,4,5], karena list sudah terurut maka hanya dilakukan satu pass tanpa pertukaran dan hasilnya tetap [1,2,3,4,5].

3. Hybrid Sort

```
1  import random
2
3  # Insertion Sort
4  def insertion_sort(arr):
5      a = arr.copy()
6      comparisons = 0
7      swaps = 0
8
9      for i in range(1, len(a)):
10         key = a[i]
11         j = i - 1
12
13         while j >= 0:
14             comparisons += 1
15             if a[j] > key:
16                 a[j+1] = a[j]
17                 swaps += 1
18                 j -= 1
19             else:
20                 break
21
22         a[j+1] = key
23
24     return a, comparisons + swaps
```

```

27 # Selection Sort (parameter) arr: Any
28 def selection_sort(arr):
29     a = arr.copy()
30     comparisons = 0
31     swaps = 0
32     n = len(a)
33
34     for i in range(n):
35         min_idx = i
36         for j in range(i+1, n):
37             comparisons += 1
38             if a[j] < a[min_idx]:
39                 min_idx = j
40
41         if min_idx != i:
42             a[i], a[min_idx] = a[min_idx], a[i]
43             swaps += 1
44
45     return a, comparisons + swaps

```

```

48 # Hybrid Sort
49 def hybridSort(theSeq, threshold=10):
50     if len(theSeq) <= threshold:
51         return insertion_sort(theSeq)
52     else:
53         return selection_sort(theSeq)
54
55 # Pengujian
56 sizes = [50, 100, 500]
57
58 print("Ukuran | Hybrid Ops | Insertion Ops | Selection Ops")
59
60 for size in sizes:
61     data = [random.randint(1, 1000) for _ in range(size)]
62
63     hybrid_ops = hybridSort(data)[1]
64     insertion_ops = insertion_sort(data)[1]
65     selection_ops = selection_sort(data)[1]
66
67     print(size, "|", hybrid_ops, "|", insertion_ops, "|", selection_ops)

```

Ukuran	Hybrid Ops	Insertion Ops	Selection Ops
50	1268	1063	1268
100	5044	5055	5044
500	125245	121548	125245

Analisis Hasil: Program membuat data angka acak dengan ukuran 50, 100, dan 500 menggunakan modul random, lalu setiap algoritma mengurutkan data tersebut sambil menghitung jumlah operasi yang terjadi dari perbandingan dan perpindahan elemen. Insertion sort mengurutkan dengan menyisipkan elemen ke posisi yang tepat, selection sort mencari nilai terkecil lalu menukarnya, sedangkan hybrid sort memilih metode berdasarkan ukuran data, yaitu jika data kecil menggunakan insertion sort dan jika data lebih besar menggunakan selection sort. Hasil run menampilkan tabel jumlah operasi dari masing-

masing algoritma sehingga dapat terlihat perbedaan efisiensi saat mengurutkan data dengan ukuran data yang berbeda.

4. Merge Tiga Sorted Lists

```
1 def mergeThreeSortedLists(listA, listB, listC):
2     i = j = k = 0
3     result = []
4
5     while i < len(listA) or j < len(listB) or k < len(listC):
6
7         a = listA[i] if i < len(listA) else float('inf')
8         b = listB[j] if j < len(listB) else float('inf')
9         c = listC[k] if k < len(listC) else float('inf')
10
11         smallest = min(a, b, c)
12
13         if smallest == a:
14             result.append(a)
15             i += 1
16         elif smallest == b:
17             result.append(b)
18             j += 1
19         else:
20             result.append(c)
21             k += 1
22
23     return result
24
25
26 # Contoh penggunaan (berbeda dari soal)
27 A = [2, 8, 15]
28 B = [1, 7, 20]
29 C = [3, 9, 14]
30
31 print(mergeThreeSortedLists(A, B, C))
```

[1, 2, 3, 7, 8, 9, 14, 15, 20]

Analisis Hasil: Fungsi `mergeThreeSortedLists` menggunakan tiga variabel indeks yaitu `i`, `j`, dan `k` untuk melacak posisi elemen pada masing-masing list, lalu selama masih ada elemen yang tersisa pada salah satu list, program akan membandingkan nilai dari ketiga list tersebut. Jika salah satu list sudah habis, nilainya diganti dengan `float('inf')` agar tidak ikut terpilih sebagai nilai terkecil. Program kemudian memilih nilai paling kecil dari ketiga list menggunakan fungsi `min()`, menambahkannya ke dalam list `result`, dan menaikkan indeks dari list yang diambil nilainya. Proses ini terus dilakukan sampai semua elemen dari ketiga list telah digabungkan. Pada contoh penggunaan dengan `A = [2, 8, 15]`, `B = [1, 7, 20]`, dan `C = [3, 9, 14]`, program akan menggabungkan dan mengurutkan semua elemen sehingga menghasilkan output `[1, 2, 3, 7, 8, 9, 14, 15, 20]`.

5. Inversions Counter

```
1  ##### Metode Brute Force (O(n^2)) #####
2  def countInversionsNaive(arr):
3      count = 0
4      n = len(arr)
5
6      for i in range(n):
7          for j in range(i+1, n):
8              if arr[i] > arr[j]:
9                  count += 1
10
11     return count
12
13  ##### Metode Merge Sort (O(n log n)) #####
14  def merge_and_count(left, right):
15      i = j = 0
16      merged = []
17      inv_count = 0
18
19      while i < len(left) and j < len(right):
20          if left[i] <= right[j]:
21              merged.append(left[i])
22              i += 1
23          else:
24              merged.append(right[j])
25              inv_count += len(left) - i
26              j += 1
27
28      merged.extend(left[i:])
29      merged.extend(right[j:])
30
31      return merged, inv_count
32
33  def merge_sort_count(arr):
34      if len(arr) <= 1:
35          return arr, 0
36
37      mid = len(arr) // 2
38
39      left, inv_left = merge_sort_count(arr[:mid])
40      right, inv_right = merge_sort_count(arr[mid:])
41
42      merged, inv_merge = merge_and_count(left, right)
43
44      return merged, inv_left + inv_right + inv_merge
45
46
47
48  def countInversionsSmart(arr):
49      _, count = merge_sort_count(arr)
50      return count
51
52  ##### Pengujian dan Pengukuran Waktu #####
53  import random
54  import time
55
56  sizes = [1000, 5000, 10000]
57
58  print("Ukuran | Naive Time | Smart Time | Inversions")
59
60  for size in sizes:
61      data = [random.randint(1, 10000) for _ in range(size)]
62
63      # Naive
64      start = time.time()
65      inv_naive = countInversionsNaive(data)
66      naive_time = time.time() - start
67
68      # Smart
69      start = time.time()
70      inv_smart = countInversionsSmart(data)
71      smart_time = time.time() - start
72
73      print(size, "|", round(naive_time,4), "|", round(smart_time,4), "|", inv_smart)
```

```
● Ukuran | Naive Time | Smart Time | Inversions
1000 | 0.028 | 0.0034 | 249234
5000 | 0.7905 | 0.0208 | 6179920
10000 | 3.519 | 0.0329 | 25300997
```

Analisis Hasil: Fungsi `countInversionsSmart` kemudian memanggil proses tersebut untuk mendapatkan jumlah inversions. Pada bagian pengujian, program membuat data acak dengan ukuran 1000, 5000, dan 10000 menggunakan modul `random`, lalu mengukur waktu eksekusi kedua metode menggunakan modul `time`. Hasil run menampilkan ukuran data, waktu yang dibutuhkan metode naive dan metode smart, serta jumlah inversions yang ditemukan. Biasanya metode merge sort akan memiliki waktu eksekusi jauh lebih cepat dibandingkan metode brute force terutama saat ukuran data semakin besar, meskipun jumlah inversions yang dihasilkan tetap sama karena keduanya menghitung nilai yang sama.